SURGESIGNAL

System Design & Architecture

A practical blueprint for an offline-aware healthcare disaster coordination platform

Purpose of this document
Help every teammate understand what we are building, how the parts connect, how emergency requests move through the system, and what we should implement first.

DESIGN BASELINE • SOFTWARE-ONLY • REQUEST-CENTRIC • OFFLINE-AWARE

The core idea

When floods or cyclones disrupt normal communication, participating hospitals and medical camps need a reliable way to raise emergency resource requests, volunteer help, and track whether supplies were actually delivered. SurgeSignal coordinates those requests through a shared, permission-controlled dashboard and an incident command centre.

Important: SurgeSignal is a coordination platform, not a hospital inventory system, clinical decision tool, or automatic dispatch engine.

1. Problem, users, and product boundaries

The problem we are solving

During a disaster, hospitals and medical camps may rely on phone calls, messaging groups, and spreadsheets. Requests can be missed, duplicated, left without an owner, or marked complete before delivery is confirmed. Network disruptions make the situation harder.

Who uses SurgeSignal?

User / role	What they do
Hospital or medical-camp user	Creates a request, views authorized open requests, and voluntarily offers resources to another facility.
Incident command-centre coordinator	Reviews requests and offers, approves or rejects offers, creates assignments, monitors unresolved needs, and verifies closure.
Logistics operator	Records transport details, dispatch, partial/full delivery, and receipt confirmation. This can be a permission assigned to an authorized user.

Product boundaries: what we will and will not build

IN SCOPE	OUT OF SCOPE
Emergency resource requests and a shared authorized request board	Mandatory, complete hospital inventory uploads
Voluntary offers from participating facilities	Automatic inventory matching or automatic allocation
Command-centre verification and assignment approval	Automatic dispatch or clinical triage
Transport, delivery, partial fulfilment, receipt confirmation	Payments or financial settlement
Offline local save, queued sync, conflict visibility, audit trail	Hardware, custom radio/satellite systems, patient medical records
Optional non-critical AI assistance	AI as a dependency for emergency workflows

Design rule: a hospital is not required to share its full inventory. A facility can simply volunteer to fulfil a request it is authorized to see.

2. High-level system architecture

Start with a modular monolith: one FastAPI backend with clearly separated internal modules. This is easier to build, test, deploy, and explain in a hackathon than microservices.

Users
Hospital / camp users
Command-centre coordinators
Logistics operators

Frontend — Responsive PWA
HTML + CSS + JavaScript
Shared dashboard and role-specific screens
Service worker + IndexedDB for offline use

Backend — FastAPI modular monolith
Authentication & permissions | Requests | Offers & assignments
Logistics & delivery | Offline synchronization | Audit & dashboard

Database — PostgreSQL
Authoritative server-side records, relationships, status changes, and audit events

How the pieces work together

- **Frontend:** displays the dashboard and forms. When offline, it can save a draft/request locally in IndexedDB and queue the operation.
- **Backend:** authenticates the user, checks role and facility permissions, validates workflow rules, and processes updates.
- **PostgreSQL:** stores the official shared state. The database is the source of truth for records accepted by the server.
- **Sync service:** sends queued operations when connectivity returns, receives server acknowledgements, and pulls authorized updates.
- **Optional AI:** may summarize information or suggest possible duplicate requests when online. Core coordination must work without it.

Deployment target

Host the static PWA over HTTPS, deploy one FastAPI service, and use a managed PostgreSQL database with private access, environment-based secrets, migrations, health checks, logs, and backups. Do not introduce Kafka, Kubernetes, Redis, or microservices unless a demonstrated need appears.

3. Main workflow: from request to verified resolution

Example scenario

Hospital A needs 20 oxygen cylinders. Hospital B offers 12. The command centre verifies and approves the offer. The delivery arrives with 10 cylinders first, so the request remains partially fulfilled. A second source supplies the remaining 10. The request closes only after the required quantity and receipt are verified.

Step	What happens	System rule
1. Submit request	Hospital A creates a request for 20 units and selects urgency.	If online, server validates and confirms it. If offline, label it “Saved locally — awaiting synchronization.”
2. Acknowledge	An authorized coordinator takes ownership of the request.	Record actor and timestamp; show unacknowledged/overdue requests.
3. Volunteer	Hospital B offers 12 units; another facility may offer the remainder.	An offer is a proposal, not an approved commitment.
4. Verify and assign	Command centre checks the offer and approves an amount.	Prevent over-allocation; a single offer cannot be assigned twice.
5. Dispatch and deliver	Logistics records transport, dispatch, quantities delivered, and receipt.	Support multiple deliveries and partial fulfilment.
6. Resolve	Coordinator verifies the full requirement is met and received.	Do not close the request merely because a vehicle was dispatched or a partial delivery arrived.

Recommended status concepts

Object	Suggested statuses / meaning
Request	Submitted → Acknowledged → In Progress → Resolution Confirmed. Also support Cancelled if authorized and justified.
Offer	Pending Review → Approved / Rejected / Withdrawn. Approval should record the approved quantity.
Assignment	Approved → Preparing → Dispatched → Partially Fulfilled / Fulfilled, with cancellation or exception handling where appropriate.
Delivery	Recorded against an assignment, with delivered quantity, time, and receipt confirmation status.

Keep statuses separate for requests, offers, assignments, and deliveries. One request can have several offers, assignments, and deliveries.

4. Database design (first-pass ERD)

This is a conceptual starting point. Exact columns, constraints, and migrations should be finalized before implementation.

Entity	Important fields	Relationship / purpose
Facility	id, name, type, location, verification_status	One facility has many users and can create requests or offers.
User	id, facility_id, role, password_hash, active_status	Belongs to a facility; actions are permission-checked.
EmergencyRequest	id, created_by, facility_id, resource_type, quantity, urgency, status, timestamps	One request can receive many offers and have many assignments.
VolunteerOffer	id, request_id, offering_facility_id, quantity_offered, status, notes	A facility volunteers a quantity against a request.
Assignment	id, request_id, offer_id, quantity_committed, responsible_party, status	An approved offer creates a committed allocation; multiple assignments can fulfil one request.
Delivery	id, assignment_id, delivered_quantity, dispatch_time, received_at, receipt_status, transport_details	An assignment can have multiple deliveries; supports partial fulfilment.
AuditEvent	id, actor_user_id, entity_type, entity_id, action, timestamp, details	Records important actions and state changes for accountability.
SyncOperation (ledger)	operation_id, client_id, user_id, result, processed_at	Makes retried offline operations idempotent and records server processing results.

Critical integrity rules

- A request's approved/committed quantity must never exceed its required quantity, unless the product explicitly permits and records an exception.
- Do not assign an offer that is rejected, withdrawn, or already fully committed.
- Use database transactions and constraints/locking when approving assignments to prevent two coordinators from over-allocating at the same time.
- A partial delivery updates fulfilled quantity but does not automatically resolve the request.
- Keep an audit trail for approvals, rejections, dispatches, delivery confirmations, cancellations, and resolution.
- Do not store patient-identifiable information; use facility and operational details only.

5. API design (proposed v1)

Use versioned REST endpoints. Each endpoint must enforce authorization on the server; hiding a button in the frontend is not a security control.

Endpoint	Purpose
POST /api/v1/auth/login	Authenticate user and establish a secure session/token.
GET /api/v1/requests	List authorized requests with filters and pagination.
POST /api/v1/requests	Create a request; validate quantity, urgency, and facility permissions.
GET /api/v1/requests/{id}	Fetch request details and permitted related records.
POST /api/v1/requests/{id}/acknowledge	Coordinator acknowledges/owns a request.
POST /api/v1/requests/{id}/offers	Submit a voluntary offer for a request.
POST /api/v1/offers/{id}/approve	Approve an offer quantity and create/confirm assignment transactionally.
POST /api/v1/offers/{id}/reject	Reject an offer with actor, time, and optional reason.
POST /api/v1/assignments/{id}/dispatch	Record dispatch and logistics details.
POST /api/v1/assignments/{id}/deliveries	Record a delivery, including partial quantity and receipt details.
POST /api/v1/requests/{id}/resolve	Resolve only after the server validates fulfilment and receipt rules.
POST /api/v1/sync/push	Submit queued offline operations using unique operation IDs.
GET /api/v1/sync/pull	Fetch authorized changes after a cursor/version.
GET /api/v1/dashboard/summary	Return aggregated operational counts for the dashboard.

Use pagination for request lists, indexes for common filters (status, urgency, facility, updated time), and consistent error responses for validation, permission, and conflict failures.

6. Offline-first behaviour and synchronization

Offline mode must be honest: saving on one device does not mean the command centre or other hospitals have received the request.

Situation	Required behaviour
User creates while online	Send to server; show confirmed status only after server acknowledgement.
User creates while offline	Save locally in IndexedDB; mark “Saved locally — awaiting synchronization.” Keep it in a visible pending queue.
Connection returns	Retry queued operations. Each operation carries a unique ID so retries do not create duplicates.
Server accepts operation	Return an acknowledgement and server record/version. Only then mark that operation synchronized.
Server rejects/conflicts	Show a clear reason and next action. Do not silently overwrite critical workflow changes.
Client pulls updates	Fetch authorized changes using a cursor or last-seen version; update the local cache.

Synchronization rules

- Queue dependent operations in order; for example, an offer cannot be sent until its request has a valid server identity.
- Use stable client-generated IDs/UUIDs and a server-side idempotency ledger.
- Use record versions or expected-version checks for important state transitions.
- Never use blind last-write-wins for approvals, quantities, assignments, or delivery confirmations.
- Background sync is not guaranteed in every browser; also retry when the app becomes active and connectivity returns.
- Offline users cannot see new changes made by other disconnected devices until they synchronize.

Minimum offline scope for the hackathon

Prioritize locally saving a new request, displaying pending synchronization state, retrying safely, and pulling server updates. Avoid promising a fully collaborative, real-time shared dashboard during a total network outage.

7. Security, reliability, and implementation plan

Security and access control

- Use verified facility accounts and backend-enforced role-based access control (RBAC).
- Check both user role and facility-level access for every sensitive operation.
- Hash passwords using a modern password-hashing library; never store plaintext passwords.
- Use HTTPS, secure session/token handling, environment variables for secrets, and least-privilege database access.
- Minimize data cached on devices; clear sensitive cached data on logout where appropriate.
- Log important actions, but do not put secrets or patient-identifiable information in logs.
- Use database migrations and regular backups; test restore procedures if time permits.

Suggested backend structure

```
surgesignal/
app/
main.py
api/ # auth, requests, offers, assignments, deliveries, sync, dashboard
core/ # config, security, permissions
models/ # SQLAlchemy entities
schemas/ # Pydantic request/response schemas
services/ # workflow rules and business logic
repositories/ # database access helpers, if useful
db/ # session, migrations
tests/
requirements.txt
README.md

frontend/
index.html
pages/
css/
js/
manifest.json
service-worker.js
```

Build order

Phase	Deliverable
1. Lock the workflow	Agree statuses, permitted transitions, role ownership, and closure rules.
2. Finalize the ERD	Define keys, foreign keys, quantity constraints, status fields, and audit events.
3. Define API contracts	Agree request/response schemas, errors, authorization, and pagination.
4. Build core online workflow	Create request → offer → approval/assignment → delivery → verified resolution.
5. Add dashboard and audit	Show open, overdue, partially fulfilled, and unresolved requests.
6. Add offline sync	Local pending queue, idempotent push, acknowledgements, conflict handling, pull cursor.
7. Test demo scenarios	Partial fulfilment, two offers racing, duplicate sync retry, permission denial, offline request.

Hackathon scope advice: finish a reliable end-to-end request workflow first. Offline sync is a differentiator, but it must not make the core workflow unreliable. Optional AI comes last.

8. Demo scenario and team decision checklist

Recommended demo sequence

- Hospital A signs in and submits a request for 20 oxygen cylinders.
- The dashboard shows the request as server-confirmed (or, in the offline demo, clearly marked pending sync).
- The command centre acknowledges the request.
- Hospital B offers 12 cylinders; Hospital C offers 8.
- The coordinator approves the offers and creates assignments.
- The logistics operator records dispatch and a first partial delivery of 10 cylinders.
- The dashboard shows 10 fulfilled and 10 still outstanding; the request remains open.
- The second delivery is recorded and receipt is confirmed.
- The coordinator resolves the request. The audit trail shows who performed each important action.

Decisions the team should confirm before coding

Decision	Proposed baseline
Product scope	Request-centric coordination; no mandatory inventory, payments, patient records, or automatic dispatch.
Roles	Facility user, command-centre coordinator, logistics permission.
Backend	FastAPI modular monolith.
Database	PostgreSQL.
Frontend/offline	HTML/CSS/JS PWA, service worker, IndexedDB.
Official data state	Server-confirmed records are authoritative; local pending records are explicitly labelled.
Approval model	Command centre verifies offers and approves quantities.
Closure rule	Close only after required fulfilment and receipt confirmation.
Offline sync	Idempotent operation IDs, explicit acknowledgements, pull cursor, visible conflicts.

One-sentence product definition

SurgeSignal is an offline-aware, request-centric coordination platform that helps authorized hospitals and medical camps request emergency resources, volunteer support, and track command-centre-approved assignments through verified delivery and resolution.

Document status: proposed architecture baseline for team review. Treat the state machine, database constraints, and synchronization contract as design items to finalize together before implementation.